

Course code : **CSE3009**
Course title : **No SQL Data Bases**
Module : **2**
Topic : **4**

Document Data Model

Objectives

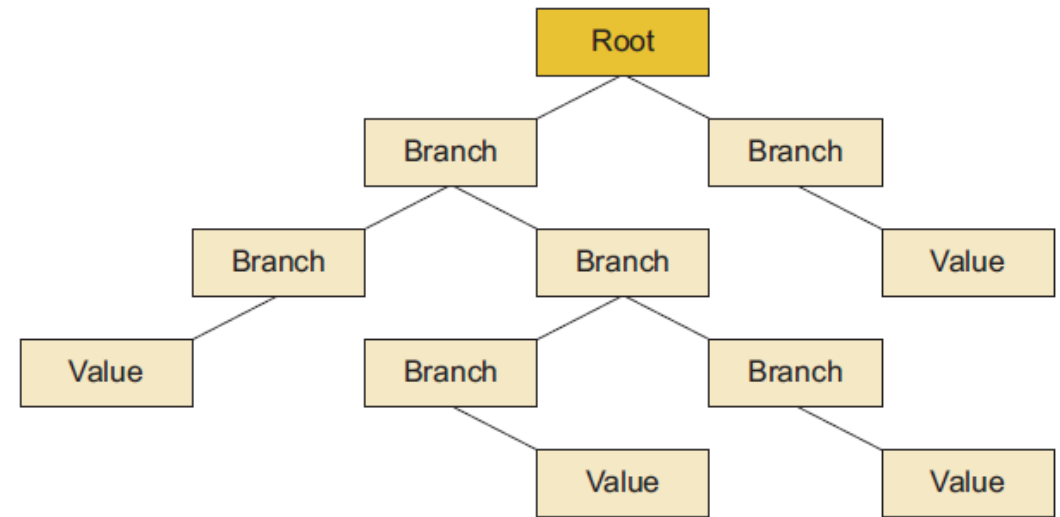
This session will give the knowledge about

- Document Data Model

Document Data Model

Document stores **use a tree structure that begins with a root node**, and have sub branches that can also contain sub-branches.

The **actual data values are usually stored at the leaf levels** of a tree.



Document Data Model

Documents are the main concept in document databases.

The database stores and retrieves documents, which can be XML, JSON, BSON, and so on.

These documents are self-describing, hierarchical tree data structures which can consist of maps, collections, and scalar values.

```
<Key=CustomerID>

{
  "customerid": "fc986e48ca6" ← Key
  "customer":
  {
    "firstname": "Pramod",
    "lastname": "Sadhalage",
    "company": "ThoughtWorks",
    "likes": [ "Biking", "Photography" ]
  }
  "billingaddress":
  { "state": "AK",
    "city": "DILLINGHAM",
    "type": "R"
  }
}
```

Document Data Model

The documents stored are similar to key-value data store but do not have to be exactly the same.

Document-Oriented NoSQL DB stores and retrieves data as a key value pair but the value part is stored as a document.

The document is stored in JSON or XML formats.

Each document contains semi-structured data that can be queried against using various query and analytics tools of the DBMS

Document Data Model VS RDBMS

Col1	Col2	Col3	Col4
Data	Data	Data	Data
Data	Data	Data	Data
Data	Data	Data	Data

Document 1

```
{
  "prop1": data,
  "prop2": data,
  "prop3": data,
  "prop4": data
}
```

Document 2

```
{
  "prop1": data,
  "prop2": data,
  "prop3": data,
  "prop4": data
}
```

Document 3

```
{
  "prop1": data,
  "prop2": data,
  "prop3": data,
  "prop4": data
}
```

MongoDB Example

```
>db.mycol.find(
  { $and: [
    {key1: value1},
    {key2:value2}
  ] } )
```

```
> db.cse.find({ $and: [{sid:
{$gt:500}}, {mark:{$gte:70}} ] } )
```

Document collections

Most document stores **group documents together in collections**. These collections look like a directory structure OS file systems.

Document collections can be used in many ways to manage large document stores.

Document collections can serve as ways to

- navigate document hierarchies,
- logically group similar documents, and
- store business rules such as permissions, indexes, and triggers.

Collections can contain other collections and trees can contain subtrees.

Document collections

XML Document Example

```
<artist>
  <artistname>IronMaiden</artistname>
  <albums>
    <album>
      <albumname>The Book of Souls
    </albumname>
    <datereleased>2015</datereleased>
    <genre>Hard Rock</genre>
    </album>
  </albums>
</artist>
```

JSON Document Example

```
{
  '_id' : 1,
  'artistName' : { 'Iron Maiden' },
  'albums' : [
    {
      'albumname' : 'The Book of Souls',
      'datereleased' : 2015,
      'genre' : 'Hard Rock'
    }
  ]
}
```

Application collections

In some situations, the collection in a document store is **used as a container for a web application package**.

This packaging format, called a **xar file**, is similar to a Java JAR file or a WAR file on Java web servers.

Packaged applications **can contain scripts as well as data**. They're loaded into the document store and use packaging tools (scripts) to reload the data.

These packaging features **make document stores more versatile, expanding their functionality to become application servers as well as document stores**.

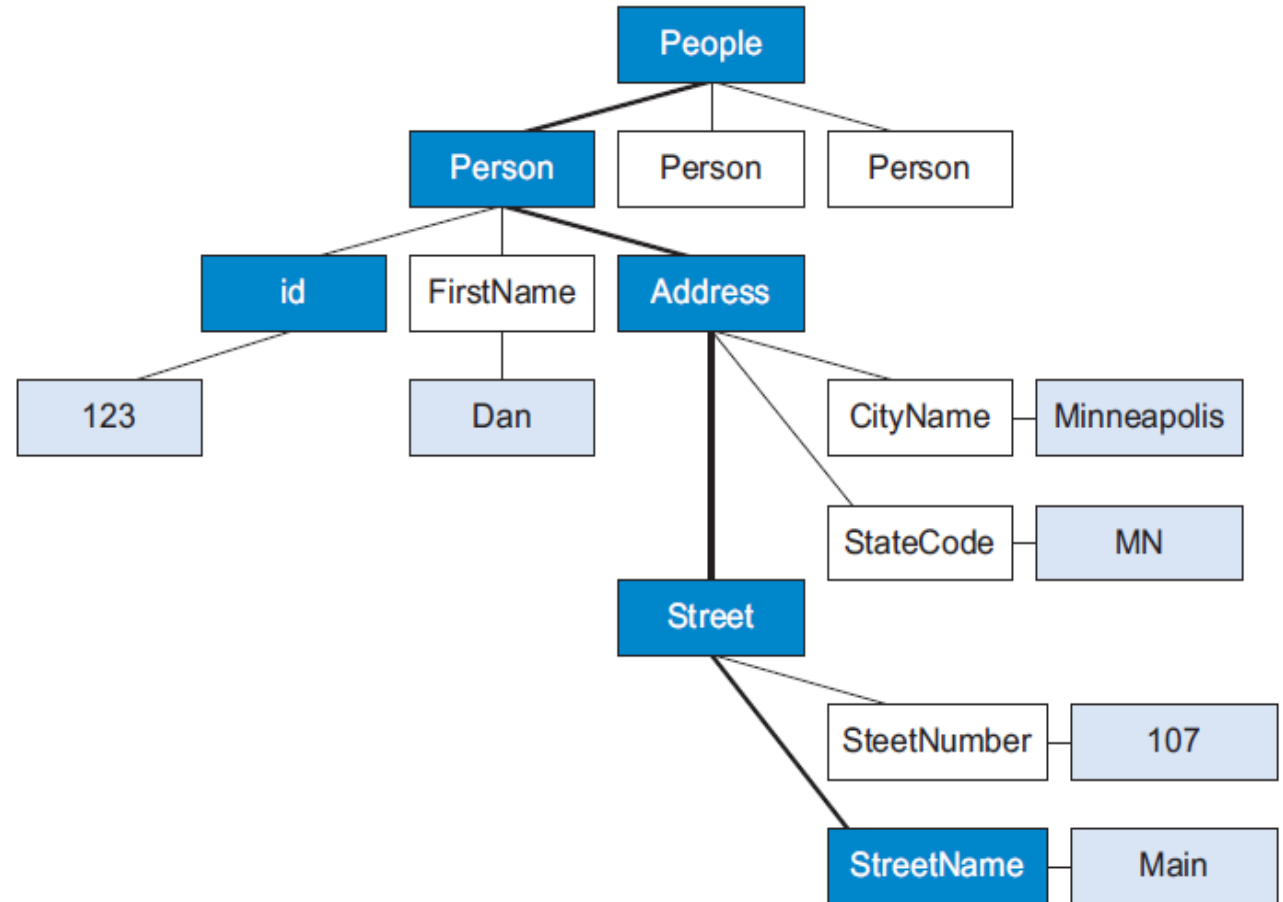


Document Data Model – Data Retrieval

How a document path is used like a key to get the value out of a specific cell in a document?

In this example, the path to the street name is

People/Person[id='123']/Address/Street/StreetName/text().



Document Store APIs

Each document store has an API or query language that specifies the path or path expression to any node or group of nodes.

For example, to select the seventh person in a list of people, you might specify this query: Person[7].

Next to look in the Address section, the full path name to the street name is the following: People/Person[id='123']/Address/Street/ StreetName/text().

Document databases such as MongoDB provide a rich query language and constructs such as database, indexes etc allowing for easier transition from relational databases.

Document store implementations

A document store can come in many varieties.

Some are based on simple serialized object trees and some are more complex, containing content that might be found in web pages with text markup.

Simpler document structures are often associated with serialized objects and may use the **JavaScript Object Notation (JSON)** format.

In our context we refer to

- **Serialized object stores** - JSON data stores as and
- **Native format document stores** - support complex content as true document stores with XML.

Benefit of the Document Model over Key-Value Stores

One benefit that document store databases have over key-value databases, is that **you can query the data itself**.

You can query against the structure of the document, as well as the elements within that structure. Therefore, **you can return only those parts of the document that you require**.

With a key-value database, you get the whole value – no matter how big (and seemingly structured) it might be. You can't query within the value.

Document Store suitable for

Document store databases are preferable for:

- Web Applications
- User Generated Content
- Catalog Data
- Gaming
- Networking/computing
- E-commerce platforms
- Content management systems
- Analytics platforms
- Blogging platforms

Document Store Not suitable for

The document type is mostly used for CMS systems, blogging platforms, real-time analytics & e-commerce applications.

Document store NoSQL databases are **not the right choice if you have to run complex search queries or if your application requires complex multiple operation transactions.**

Here are examples of some of the leading document store DBMSs.

- MongoDB, DocumentDB, CouchDB, MarkLogic, OrientDB

Case study: ad server with MongoDB

Do you ever wonder how those banner ads show up on the web pages you browse or how they really seem to target the things you like or are interested in?

It's done with [ad serving](#).

The original reason for MongoDB, was to create a service that would quickly send a banner ad to an area on a web page for millions of users at the same time.

MongoDB can be used in some of the following use cases:

- Content management
- Real-time operational intelligence
- Product data management
- User data management.

Summary

This session will give the knowledge about

- Document Data Model